

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Software Engineering 2 - DD

Author: **Simone Controguerra**

Author: **Matteo Delogu**

Author: **Gianluca Di Paola**

Advisor: Prof. Elisabetta Di Nitto
Academic Year: 2025-2026

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms and Abbreviations . . . 1
1.4 Revision History . . . 3
1.5 Reference Documents . . . 3
1.6 Document Structure . . . 3

2 Architectural Design **5**
2.1 Overview: High-level components and their interaction . . . 5
2.1.1 Presentation Tier . . . 5
2.1.2 Application Tier . . . 5
2.1.3 Data Tier . . . 6
2.2 Component View . . . 7
2.2.1 Client . . . 7
2.2.2 Application Server . . . 8
2.3 Deployment View . . . 10
2.4 Runtime View . . . 12
2.4.1 User registration . . . 12
2.4.2 Registered User login . . . 13
2.4.3 Registered User deletion . . . 14
2.4.4 Registered User records a Trip . . . 15
2.4.5 Registered User sees a past Trip . . . 16
2.4.6 Registered User inserts a Bike Path in Manual Mode . . . 17
2.4.7 Registered User inserts a Bike Path in Automatic Mode . . . 18
2.4.8 User searches a Bike Path . . . 19

2.5 Component Interfaces 20
2.5.1 Fat Client Interfaces 20
2.5.2 Application Server 21
2.6 Selected Architectural Styles and Patterns 23
2.6.1 Load Balancing 23

3 User Interface Design **25**

4 Requirements traceability **27**

5 Implementation, Integration and Test plan **31**
5.1 Component Priority 31
5.2 Integration and Testing Plan 32
5.2.1 DBMS and Data Access Module 32
5.2.2 User Manager 33
5.2.3 Trip Manager and Bike Path Manager 33
5.2.4 Connector Module 34
5.2.5 Report Manager 35
5.2.6 Street Manager 35
5.2.7 Path Recommendation Engine 36
5.2.8 Sensor Manager 37
5.2.9 Trip Detector 37
5.2.10 Metrics Engine 38
5.2.11 External API Manager 39
5.2.12 Offline Sync Module 39
5.3 System Testing 40

6 Effort spent **41**

7 References **43**

8 Use of Generative AI **45**

1 | Introduction

1.1. Purpose

Nowadays self-care activities have become really popular such as keeping themselves fit, in particular a lot of people ride their bike either to stay healthy or to adopt an eco-friendly means of transport. Although, streets cannot be the safest place for bikes since cars can run very fast there. The solution is to ride on designated bike paths or safe roads where cars are few and they cannot run too fast.

Best Bike Paths is a service that offers an innovative solution to bikers that want to ride safely. Its purpose is whether to give the opportunity to its users to record their personal trips and store them as an archive of sport activities, enriched with different kinds of related information, or to provide suggestions for safe and efficient bike paths to reach a given destination, highlighting the better ones. Best Bike Paths is a service with community features, that aims to build a network of bike paths for everyone.

1.2. Scope

Best Bike Paths is a system that collects and processes cycling data through the coordination of software components. Its scope includes managing registered users and their personal trips, acquiring data from mobile devices' sensors, and producing processed information such as metrics and automatically inferred bike paths. The system maintains a shared collection of bike paths, updated through user reports, ensuring that public streets remain accurate and reliable. Best Bike Path provides a search service that allows the users to find suitable paths for their destination.

1.3. Definitions, Acronyms and Abbreviations

- **BBP (Best Bike Paths):** The name of the system.
- **Bike Path:** A route specifically designated or suitable for bicycle travel, which can

be either manually defined by users or automatically generated by the system. It is formally defined as a set consisting of: a Starting Point, a Finishing Point, and an ordered sequence of traversable Streets (see definition of Street).

- **Trip:** A standalone, recorded instance of a user traveling from one point to another, which includes sensor data, metrics, and potentially weather information.
- **Street:** A segment of road suitable for cycling (factors taken into account include average bike speed limit, roadway width, traffic volume, dedicated bike lanes, etc.) that forms part of a bike path, with attributes such as status and presence of irregularities (e.g., potholes).
- **Navigation System:** An external service or API used by the system to evaluate road safety for bikers, considering factors such as bike track availability and traffic conditions.
- **Weather-forecast System:** An external service or API used by the system to provide weather information associated with a trip.
- **GPS (Global Positioning System):** A satellite-based navigation system that provides location and time information, in all weather conditions, anywhere on Earth.
- **IMU (Inertial Measurement Unit) Sensors:** Electronic devices that measure and report a physical object's specific force, angular rate and orientation, using a combination of both accelerometer and gyroscope.
- **API (Application Programming Interface):** A set of protocols and tools that enable different software applications to communicate with each other.
- **JSON (JavaScript Object Notation):** A lightweight data-interchange format that is easily readable and writable by both humans and machines.
- **UI (User Interface):** The interface through which users interact with the platform, encompassing buttons, screens, and navigation elements.
- **TTL (time-to-live):** Maximum duration for a cache entry to be considered valid.
- **Redundancy:** The inclusion of supplementary components or systems to ensure reliability and availability in the event of a primary component failure.
- **Scalability:** The system's capacity to manage a growing number of users or increased load by incorporating additional resources without significant degradation in performance.

- **Uptime:** The period during which a system is fully operational and accessible to users without interruption.
- **Backup:** The process of duplicating and archiving data to ensure restorability in the event of loss or damage to the original data.

1.4. Revision History

- **v1.0** First version of the document, revised on 6th of January 2026.

1.5. Reference Documents

Professor Elisabetta Di Nitto, *Slides of the course "Software Engineering 2"* at PoliMi. Polytechnic University of Milan, *Specifications of the DD assignment for the Software Engineering II course.*

1.6. Document Structure

This document is structured into seven main chapters.

1. **Introduction:** it provides a high-level overview of the project and defines the main terminology and acronyms used throughout the document.
2. **Architectural Design:** it describes the system architecture, from the 3-tier model to the design through multiple components.
3. **User Interface Design:** it is related to UI aspects.
4. **Requirements traceability:** it contains a detailed mapping of the RASD Requirements into design elements.
5. **Implementation, Integration and Test plan:** it outlines the development strategy where each module is implemented and tested in isolation and, then, incrementally integrated into the complete system, followed by system-level testing.
6. **Effort spent:** it indicates how the workload has been distributed among the contributors.
7. **References:** it includes references to documents and tools that have been used in the writing of the document.
8. **Use of Generative AI:** it includes eventual AI tools used for the realization of

this document.

2 | Architectural Design

2.1. Overview: High-level components and their interaction

The Best Bike Paths platform requires an architecture able to provide reliability, efficiency, scalability. For this reason, its architecture follows a 3-tier model divided in a Presentation Tier, an Application Tier, and a Data Tier. Each tier will be described through an overview below and in detail in the next sections.

2.1.1. Presentation Tier

The presentation tier is composed by a fat client which manages both the user interface and, partially, the application logic, in order to minimize the communication with the Application Server and to reduce latency for simple operations such as trip metrics' computation. It's also responsible to retrieve data from the device's sensors and of local data pre-processing such as detection of trips.

It can communicate directly with the Application Server through HTTPS and the internal APIs.

2.1.2. Application Tier

The application layer is just composed by the Application Server which is responsible for users' sessions management, data validation, shared business logic (such as Bike Paths' merging, publication and ranking), notification service, external APIs management.

It communicates through HTTPS and internal APIs with the Fat Client and it's the only one to communicate with the Data Tier.

2.1.3. Data Tier

The data tier is composed by the DBMS server which responsibility is to store permanently data about the system data (such as Trips, Bike Paths, Users, Reports and User preferences), to manage both structured data and semi-structured data, to keep information integrity.

It communicates just with the Application Server.

2.2. Component View

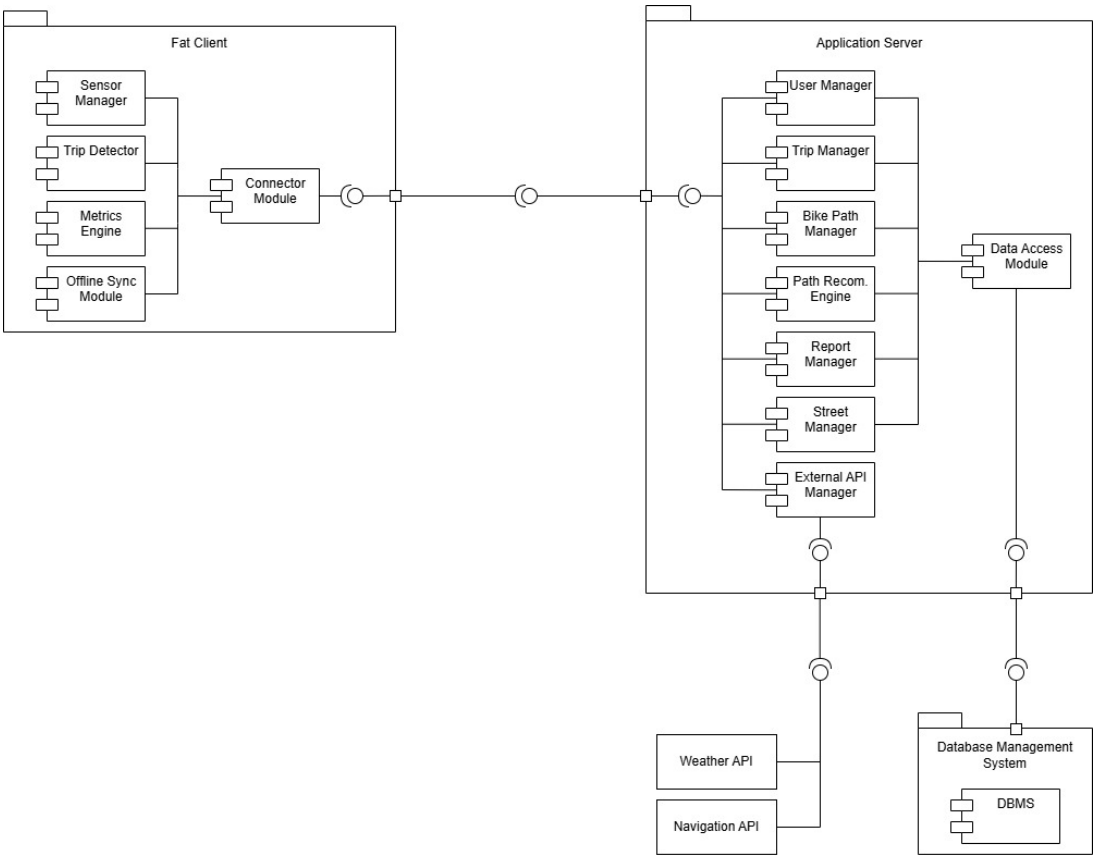


Figure 2.1: Component diagram

2.2.1. Client

Sensor Manager

This component is responsible for the communication with the device sensors and for the collection of GPS and IMU data. It directly communicates with the other modules that are in the Client layer.

Trip Detector

This component is responsible for the automatic detection of Trips based on sensor data.

Metrics Engine

This component is responsible for the computation of Trip metrics. It receives data from the Sensor Manager and from the Trip Detector modules.

Offline Sync Module

This component is responsible for the offline re-synchronization of the on-going Trip, when Internet connection is not available.

Connector Module

This component is responsible for the communication between Client and Server, it provides a REST interface.

2.2.2. Application Server

User Manager

This component is responsible for the management of users' profile, credentials and permissions.

Trip Manager

This component is responsible for the storing and the enrichment of Trips. It interacts with the External API Manager module to add weather information to the Trip, if available.

Bike Path Manager

This component is responsible for the creation, validation and publication of Bike Paths. It interacts with the External API Manager module in order to check safety of the Bike Paths. Also, it interacts with Street Manager module so that the component can propagate changes in existing Bike Paths.

Path Recommendation Engine

This component is responsible for the discovery, retrieval, scoring and ranking of Bike Paths when a user queries the system for a Bike Path to reach a destination. So, Bike Paths are retrieved, their score gets computed and they get ranked according to said score.

Report Manager

This component is responsible for receiving reports from the users and to retrieve data about Streets' conditions.

Street Manager

This component is responsible to update Streets' status according to new received reports. It interacts with the Report Manager module.

External API Manager

This component is responsible to call external APIs such as the Weather and Navigation ones, in order to enrich or validate data.

Data Access Module

This component is responsible for the communication between the Server modules and the Database.

2.3. Deployment View

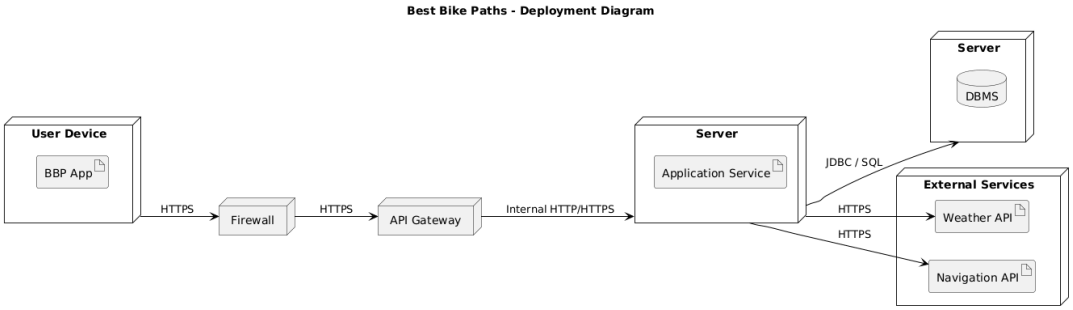


Figure 2.2: Deployment diagram

User Device

The device must be a device able to connect to Internet, download the BBP application, and it shall have GPS and IMU sensors. The BBP application collects sensor data locally and it computes part of the application logic. It communicate with the Application Server through HTTPS protocol. It uses local storage temporarily for offline operations.

Firewall

That's necessary to guarantee security for the users and the platform itself. It represents a protection for the Application Server and the DBMS Server from external attacks. Only safe HTTPS messages can reach the backend.

API Gateway

It represent the only entry point for the Client requests. It performs TLS termination and it routes REST APIs calls to the Application Server. It hides the internal architecture from any external exposures.

Application Server

This is the core part of the application, where most of the logic resides. It communicates with the Client through HTTPS protocol and RESTful APIs. It interacts with the DBMS when data is needed to be retrieved.

DBMS Server

It contains all the data used by the application, such as Trips and Bike Paths. The DBMS uses a relational database system based on PostgreSQL. It maintains the data consistency.

External Services

The platform relies on two external services, one for enrich Trips with Weather Data (if available), and one to check the suitability for Streets when any user wants to publish a Bike Path. Both the services are provided by external APIs.

2.4. Runtime View

2.4.1. User registration

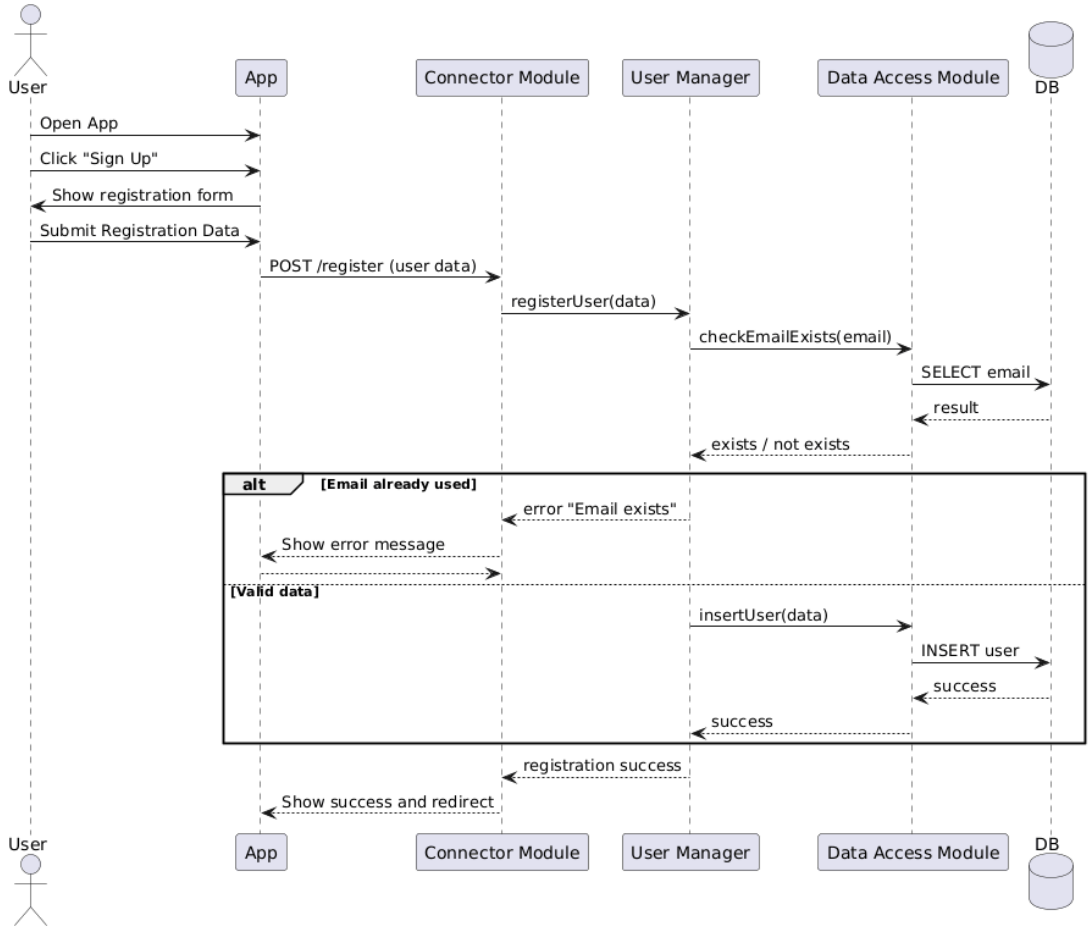


Figure 2.3: Sequence Diagram: User registration

The user interacts with the Registration page of BBP app and fills the forms. The data gets forwarded to the Connector Module of the Clients in order to send it to the Server and get managed by the User Manager module. Then, data is checked and added on the Database through the Data Access Module.

2.4.2. Registered User login

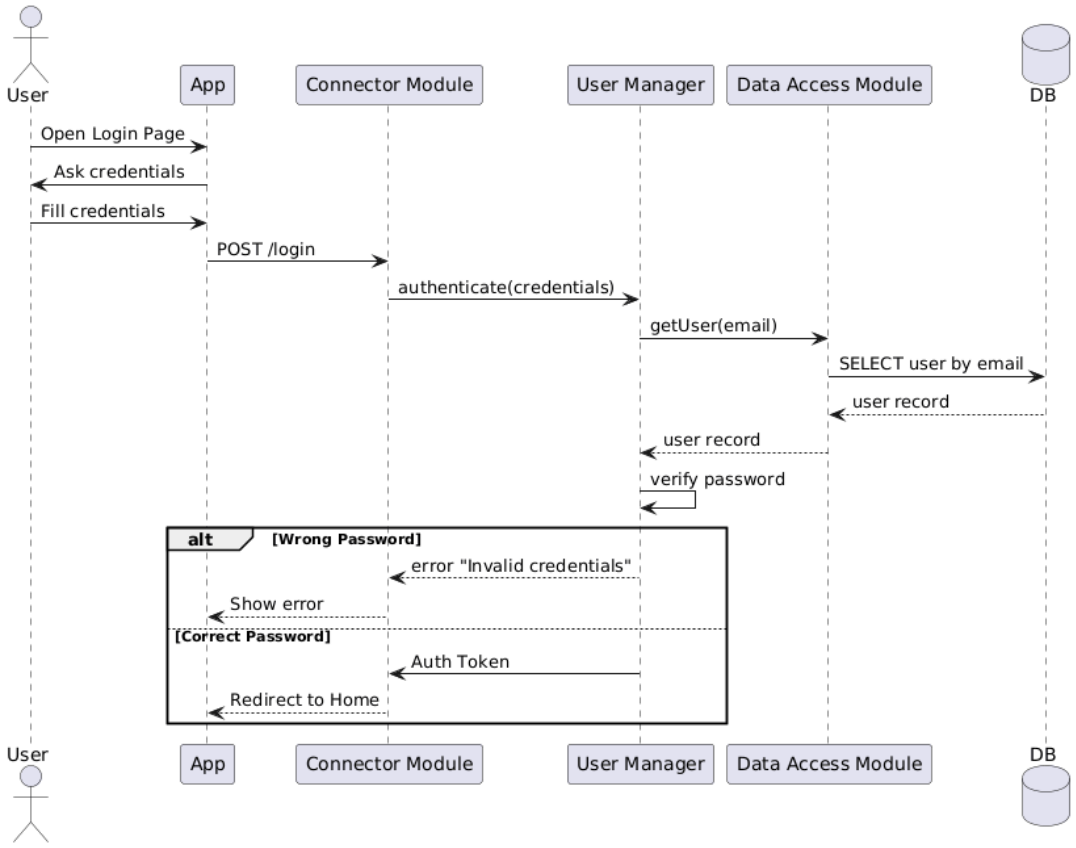


Figure 2.4: Sequence Diagram: Registered User login

The user interacts with the Login page of BBP app and fills the forms. The data gets forwarded to the Connector Module of the Clients in order to send it to the Server and get managed by the User Manager module. Then, data is checked on the Database through the Data Access Module. The outcome is then showed to the user.

2.4.3. Registered User deletion

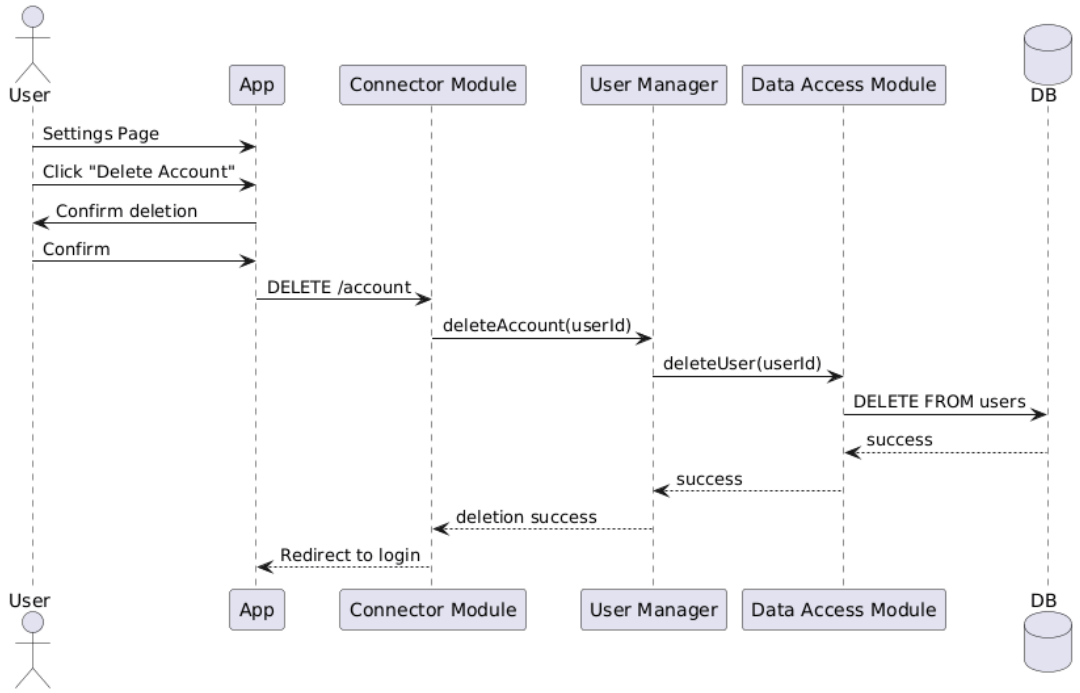


Figure 2.5: Sequence Diagram: Registered User deletion

The user interacts with the Settings page of BBP app to delete their account. Messages are forwarded to the Connector Module of the Client and they get to the User Manager module of the Server. Database operations are made through the Data Access Module to delete the information about the user requesting the deletion of their account.

2.4.4. Registered User records a Trip

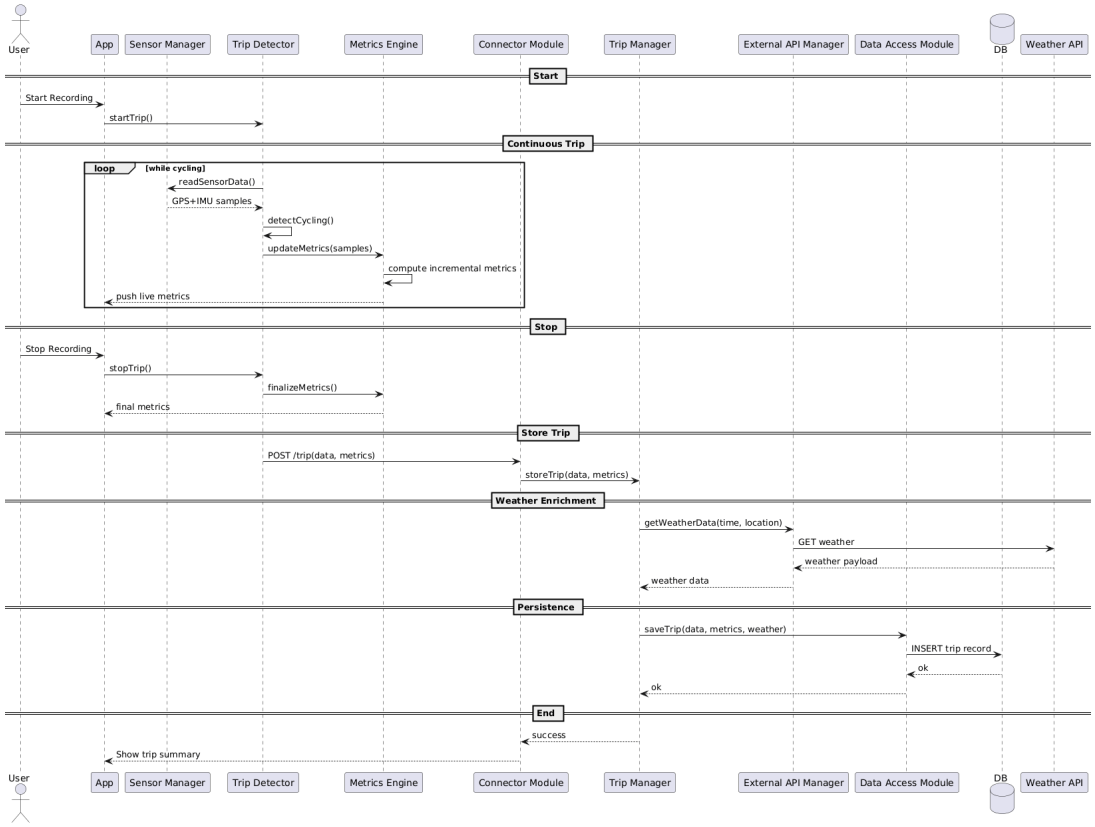


Figure 2.6: Sequence Diagram: Registered User records a Trip

The user is starting the recording of a trip. The user can start manually the recording or the Trip Detector module of the Client can automatically detect it. During the recording metrics are computed by the Metrics Engine module of the Client. Once the recording gets stopped, the final metrics are computed and the Trip is stored into the Database, through the forwarding to the Connector Module and to the Data Access Module. External API for Weather is called by the Manager module of the Server to enrich data.

2.4.5. Registered User sees a past Trip

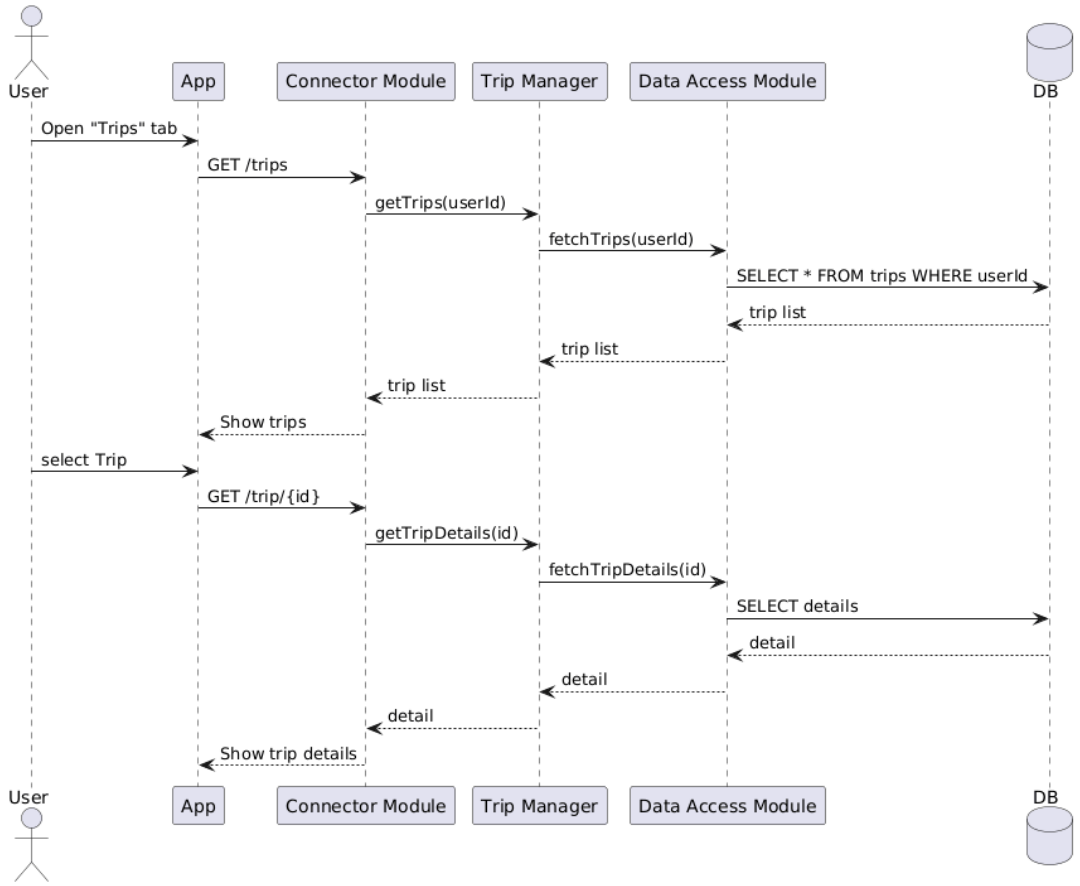


Figure 2.7: Sequence Diagram: Registered User sees a past Trip

The user interacts with the BBP app in order to see the past Trips. The Connector Module of the Client sends a request to the Trip Manager of the Server that forwards it to the Database through the Data Access Module. The collection of that user's past Trips are returned and if the user selects one of them the same iteration repeats to ask to the Database the details about that Trip.

2.4.6. Registered User inserts a Bike Path in Manual Mode

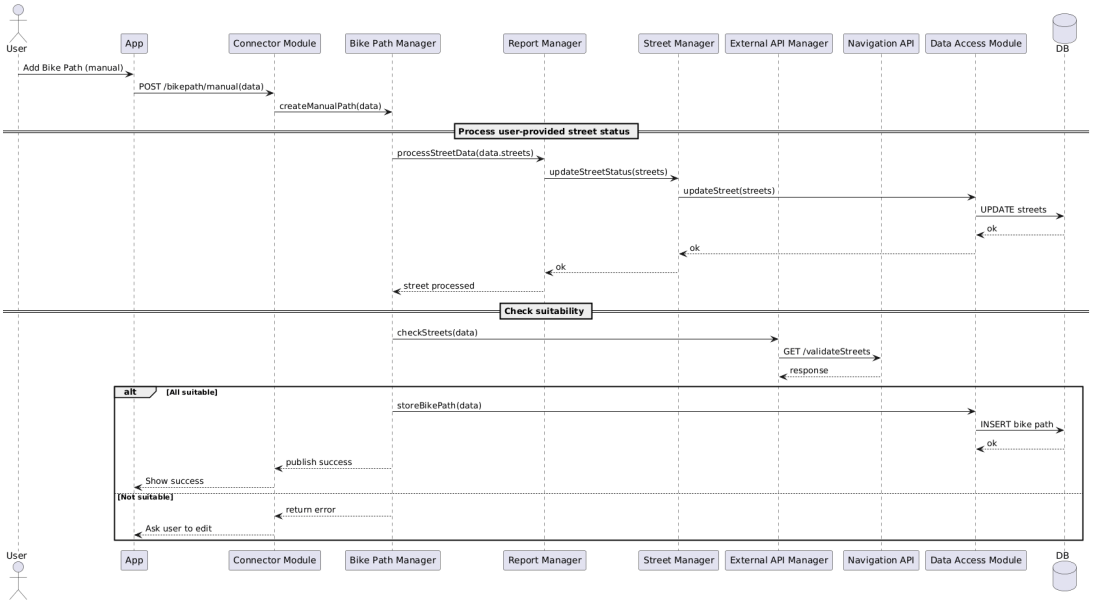


Figure 2.8: Sequence Diagram: Registered User inserts a Bike Path in Manual Mode

The user wants to publish a Bike Path inserting manually the Streets that compose the Bike Path. The data is passed from the Client to the Server through the Connector Module. On the server, the Bike Path Manager, the Report Manager and the Street Manager work to manage the status of the Streets and store the data into the Database. Meanwhile, the suitability of the Streets is also checked through the External Navitation API called by the External API Manager module of the Server.

2.4.7. Registered User inserts a Bike Path in Automatic Mode

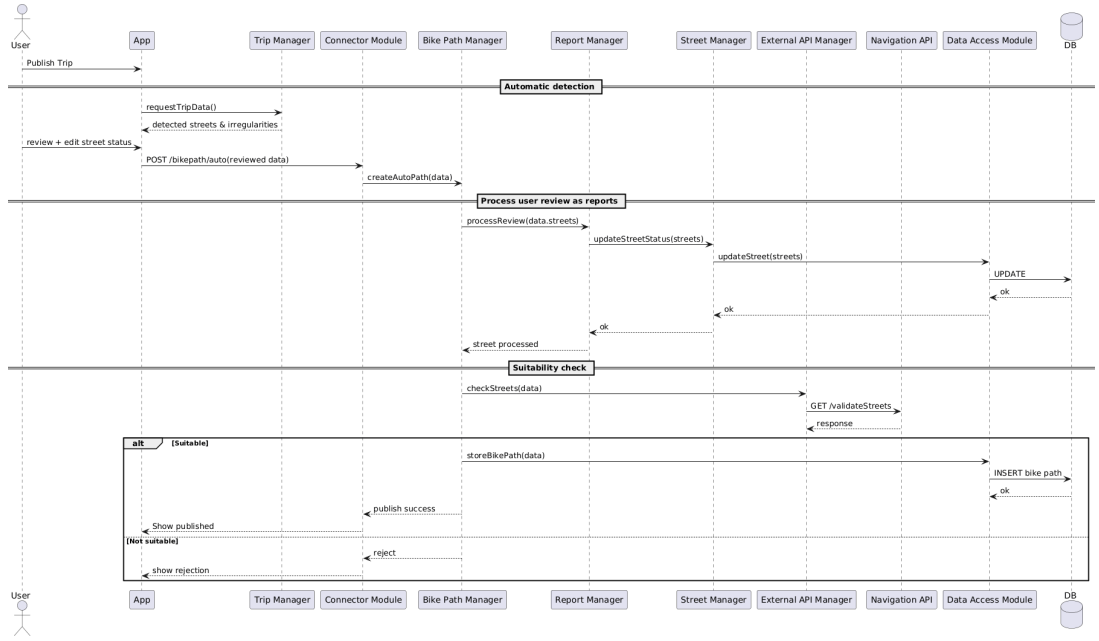


Figure 2.9: Sequence Diagram: Registered User inserts a Bike Path in Automatic Mode

The user wants to publish a Bike Path converting data from a previous Trip. So data gets collected by the Trip Manager and forwarded to the Server by the Connector Module. Before that, the user reviews the generated Bike Path and the report gets sent to the Server. Streets are checked for suitability through the External API Manager, and the Streets' status get updated trough the Report and Street Managers. Data is then stored into the Database through the Data Access Module.

2.4.8. User searches a Bike Path

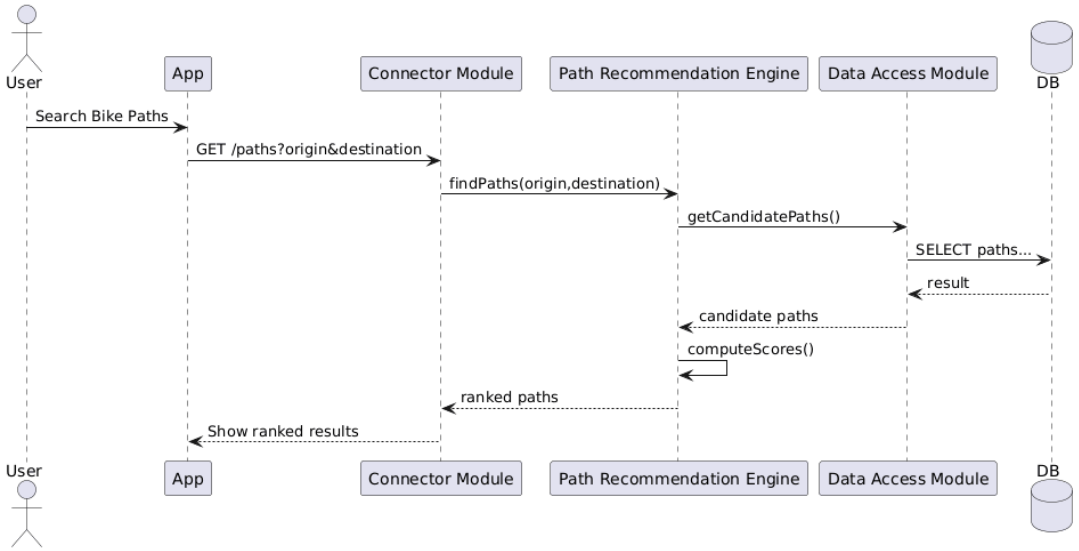


Figure 2.10: Sequence Diagram: User searches a Bike Path

The user searches for a Bike Path to reach a given destination from a given origin. The request gets forwarded to the Server through the Connector Module. Here, the Path Recommendation Engine retrieves possible candidates from the Database through the Data Access Module. Then the module computes the score of the retrieved Bike Paths and it rank to show them back to the user.

2.5. Component Interfaces

2.5.1. Fat Client Interfaces

Sensor Manager

- readSensorData()

Trip Detector

- startTrip()
- stopTrip()
- detectCycling()

Metrics Engine

- updateMetrics(SensorData: SensorData)
- finalizeMetrics()

Offline Sync Module

- storeData(JSONData: String)
- retrieveData()

Connector Module interface

- `POST /register`
  - userData=UserData
- `POST /login`
  - email=String & password=String
- `POST /logout`
  - UserID=int
- `DELETE /account`
  - UserID=int
- `POST /recover_account`

  - email=String
- `POST /trip`
  - Data=TripData & Metrics=TripMetrics
- `GET /trips`
  - UserID=int
- `GET /trip/{id}`
- `POST /bikepath/manual`
  - Data=TripData
- `POST /bikepath/auto`
  - Data=TripData
- `GET /path?origin&destination`
- `POST /update_preferences`
  - Preferences: Preferences

2.5.2. Application Server

User Manager

- registerUser(userData: UserData)
- authenticate(email: String, password: String)
- deleteAccount(userID: Int)
- recoverAccount(email: String)
- updateUserPreferences(Preferences: Preferences)

Trip Manager

- storeTrip(data: TripData, metrics: TripMetrics)
- getTrips(userID: Int)
- getTripDetails(id: Int)

Bike Path Manager

- createPath(pathData: Path)

Path Recommender Engine

- findsPath(origin: String, destination: String)

Report Manager

- processReview(streets: list<Street>)

Street Manager

- updateStreetStatus(streets: list<Street>)

External API Manager

- getWeatherData(time: Time, location: String)
- checkStreet(street: Street)

Data Access Module

- checkEmailExist(email: String)
- insertUser(userData: UserData)
- getUser(email: String)
- deleteUser(userId: Int)
- storePreferences(Preferences: preferences)
- saveTrip(data: TripData, metrics: TripMetrics, weather: WeatherData)
- fetchTrips(userId: Int)
- fetchTripDetails(id: Int)
- updateStreet(street: Street)
- storeBikePath(pathData: Path)
- getCandidatePaths(origin: String, destination: String)

2.6. Selected Architectural Styles and Patterns

BBP follows different patterns to ensure good scalability, maintainability and usability.

Client-Server Architecture

The system, as already described in this chapter, is based on a **3 tier** Client-Server architecture. This is done in order to divide responsibilities between multiple layers:

- **Presentation layer:** the mobile application manages user interfaces, logic that must run into the client and backend communication.
- **Application layer:** the core business logic is implemented on the application server and it can retrieve data from the Data layer.
- **Data layer:** a DBMS, based on postgreSQL, stores persistent data.

RESTfull architecture

The integration between the client and the application server is performed using a RESTful architecture. Its main feature is the statelessness, meaning that servers maintain no information about past requests; instead, each client request must contain all necessary context, making it easier to develop and test both frontend and backend.

MVC

The MVC pattern is used in the client application to separate user interface, application logic and local data management. This pattern ensures a clear division of responsibilities, also making the application easier to maintain. In addition it can benefit the development with it's clear division of roles.

Microservice Pattern

The application server adopts a microservice approach, splitting the business logic into independent and decoupled services as seen in **section 2.2.2**. Each **service** can be developed, tested and replicated independently, ensuring scalability and resilience.

2.6.1. Load Balancing

Load balancing is granted via the API gateway. The gateway offers a unique entry point to backend services, granting also security, rate limiting and logging.

3 | User Interface Design

The details of the UI have already been analyzed and showed in detail in the User Interface section of the RASD document.

4 | Requirements traceability

Requirement	Component
[R1] The system shall allow potential service consumers to **create accounts**.	**User Manager** (registerUser(userData)), **Connector Module** (POST /register), **Data Access Module / DBMS** (insertUser(userData))
[R2] Users shall be able to securely **login and logout** using a username-password system or third-party authentication.	**User Manager** (authenticate(email, password)), **Connector Module** (POST /login)
[R3] Users shall be able to **recover** their accounts using a password-recovery feature.	**User Manager** (recoverAccount(Id)), **Connector Module** (POST /recover_account)
[R4] The system shall present a **consent** flow to collect explicit permission for background location/activity recognition and sensor data processing, storing such decision.	**User Manager** (updateUserPreferences(preferences)), **Connector Module** (POST /update_preferences), **DBMS & Data Access Module** (storePreferences(preferences))
[R5] Registered users shall be able to configure preferred **units of measure** and language.	**User Manager** (updateUserPreferences(preferences)), **Connector Module** (POST /update_preferences), **DBMS & Data Access Module** (storePreferences(preferences))
[R6] The system shall accept user input for **origin and destination** and display all possible bike paths, ordered by score.	**Path Recommendation Engine** (retrieval, scoring, ranking), **Connector Module** (GET /path?origin&destination), **DBMS & Data Access Module** (getCandidatePaths(origin, destination))

Requirement	Component
[R7] Registered users shall be able to **manually create bike paths** (i.e.,"Manual Mode") by providing: a starting point; a finishing point; an ordered sequence of streets, specifying their overall status and the presence of irregularities (e.g., potholes).	**Bike Path Manager** (creation, validation, publication), **Report Manager** (processReview(streets)), **Street Manager** (updateStreetStatus(streets)), **External API Manager** (checkStreet(street)), **DBMS & Data Access Module** (storeBikePath(pathData), updateStreet(street))
[R8] Registered users shall be able to evaluate, modify (if necessary) and approve potential **obstacles** detected by the system in Automatic Mode for a bike path, assessing the correctness of information before publication.	**Report Manager** (processReview(streets)), **Bike Path Manager** (createPath(pathData)), **Street Manager** (updateStreetStatus(streets))
[R9] Registered users shall be able to **delete** their profiles.	**User Managr** (deleteAccount(userID)), **Connector Module** (DELETE /account), **DBMS & Data Access Module** (deleteUser(userId))
[R10] Registered users shall be able to **monitor** trip metrics.	**Metrics Engine** (updateMetrics(SensorData)), **Trip Manager** (getTrips(userID), getTripDetails(id)), **Connector Module** (GET /trips, GET /trip/{id})
[R11] Registered users shall be able to **manually start** a trip.	**Trip Detector** (startTrip(), stopTrip()), **Sensor Manager** (readSensorData()), **Metrics Engine** (updateMetrics(SensorData))
[R12] The system shall **record** personal **trips** and present statistics (distance, time, average speed) and, when available, weather data associated with the trip period.	**Trip Manager** (store and enrichment), **Metrics Engine** (updateMetrics(SensorData)), **External API Manager** (getWeatherData(time, location)), **DBMS & Data Access Module** (saveTrip(data, metrics, weather))

Requirement	Component
[R13] The system shall allow offline trip recording (street-queuing data et al) and automatically **sync** when connectivity is restored.	**Offline Sync Module** (storeData(JSONData), retrieveData()), **Connector Module** (POST /trip), **DBMS & Data Access Module** (saveTrip(data, metrics, weather))
[R14] The system shall be able to **automatically create bike paths** (i.e., "Automatic Mode") by processing data from the sensors of the device of the user (GPS, accelerometer, gyroscope).	**Sensor Manager** (gather GPS/IMU data with readSensorData()), **Trip Detector** (detectCycling()), **Bike Path Manager** (createPath(pathData)), **DBMS & Data Access Module** (storeBikePath(pathData))
[R15] The system shall be able to **save** manually created bike paths, allowing registered users to create bike paths in "Manual Mode".	**Bike Path Manager** (createPath(pathData)), **DBMS & Data Access Module** (storeBikePath(pathData))
[R16] The system shall support **auto-pause/auto-resume** when the user stops/moves again below/above thresholds.	**Trip Detector** (detectCycling()), **Metrics Engine** (updateMetrics(SensorData))
[R17] The system shall verify the **compliance of streets**, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed limit, roadway width, traffic volume, dedicated bike lanes).	**External API Manager** (checkStreet(street)), **Bike Path Manager** (createPath(pathData)), **Street Manager** (updateStreetStatus(streets))
[R18] The system shall prevent registered users from publishing bike paths with **unallowed** streets.	**Bike Path Manager** (createPath(pathData)), **External API Manager** (checkStreet(street))
[R19] The system shall **merge multiple contributions** related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.	**Street Manager** (updateStreetStatus(streets)), **Report Manager** (processReview(streets)), **Bike Path Manager** (createPath(pathData))

Table 4.1: Requirements Traceability

5 | Implementation, Integration and Test plan

This chapter includes the details regarding the implementation, integration and testing process. In particular, it shows the priority for each single component, regarding both the implementation and the integration. The chosen approach will follow a bottom-up strategy, starting from the individual modules so that the developers can focus on them singularly. Each component will be tested individually and, since the components interact with each other, drivers will be used to simulate the other participant modules. After that each module will be implemented and tested, the process will continue with the integration of the single parts in order to compose the final and complete system, ready for general testing.

5.1. Component Priority

The priority is chosen according to the essentiality of the module and to the interaction between modules, so that core functionalities and critical dependencies get implemented before those feature that enhance the user experience or that enrich the platform features.

- **Sensor Manager**: high priority.
- **Trip Detector**: medium priority.
- **Metrics Engine**: medium priority.
- **Offline Sync Module**: low priority.
- **Connector Module**: very high priority.
- **User Manager**: very high priority.
- **Trip Manager**: very high priority.
- **Bike Path Manager**: very high priority.

- **Path Recommendation Engine**: high priority.
- **Report Manager**: high priority.
- **Street Manager**: high priority.
- **External API Manager**: low priority.
- **Data Access Module**: very high priority.

This prioritization ensures that core functionalities are implemented first

5.2. Integration and Testing Plan

The development process of the individual modules is structured following an order which allows the other components to be integrated as soon as possible, so that an early testing process will be guaranteed.

5.2.1. DBMS and Data Access Module

The first step is to implement the DBMS and the module responsible for the communication between the server and the database, since all the components interact with it and depend on it. After its implementation, the database shall be tested in order to guarantee reliability. If so, integration of other components can happen.

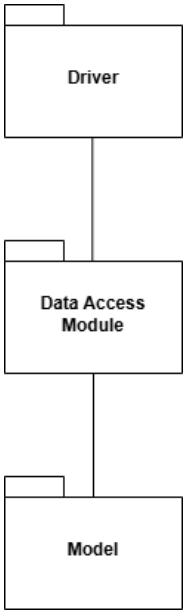


Figure 5.1: Implementation step: 1

5.2.2. User Manager

Once the database has been implemented and tested, the platform can't work without users and their interactions, so the next component to implement and to test is the User Manager. It will once be tested individually and then integrated with the DBMS.

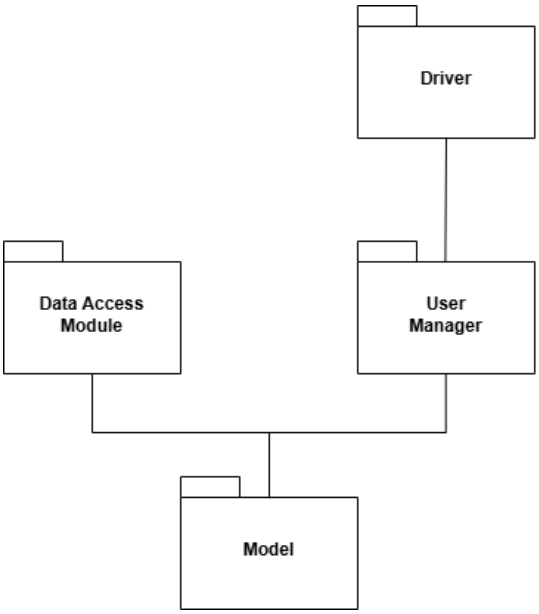


Figure 5.2: Implementation step: 2

5.2.3. Trip Manager and Bike Path Manager

Trips and Bike Paths are the the key elements of the BBP platform. They will be implemented individually and in parallel, since they are equally important. They will be tested singularly first, then they will be integrated and tested together.

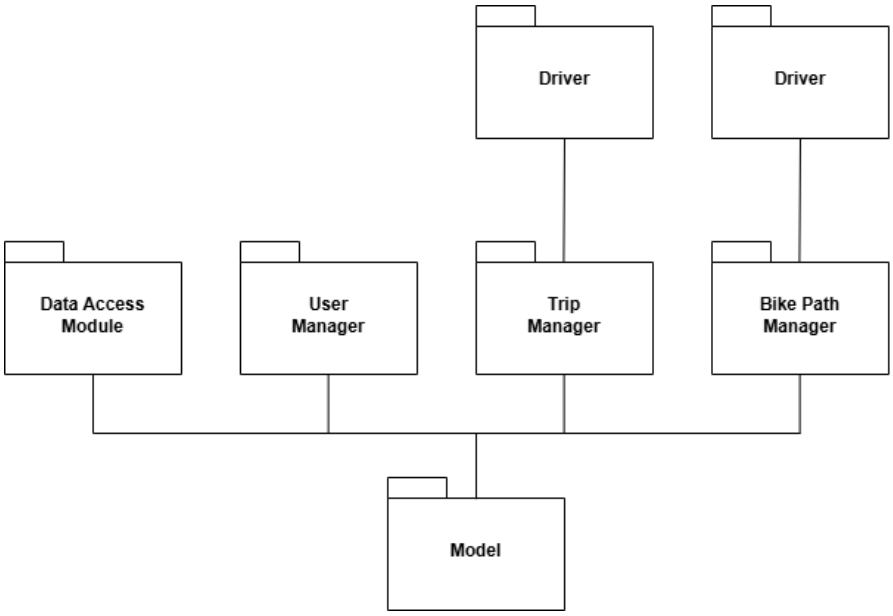


Figure 5.3: Implementation step: 3

5.2.4. Connector Module

Part of the logic resides on the client, this component is essential to switch message in a secured way between the mobile device and the Application Server. It must be implemented and tested individually using some drivers that simulate the messages sent from the client, then it will be integrated with the system and be tested altogether.

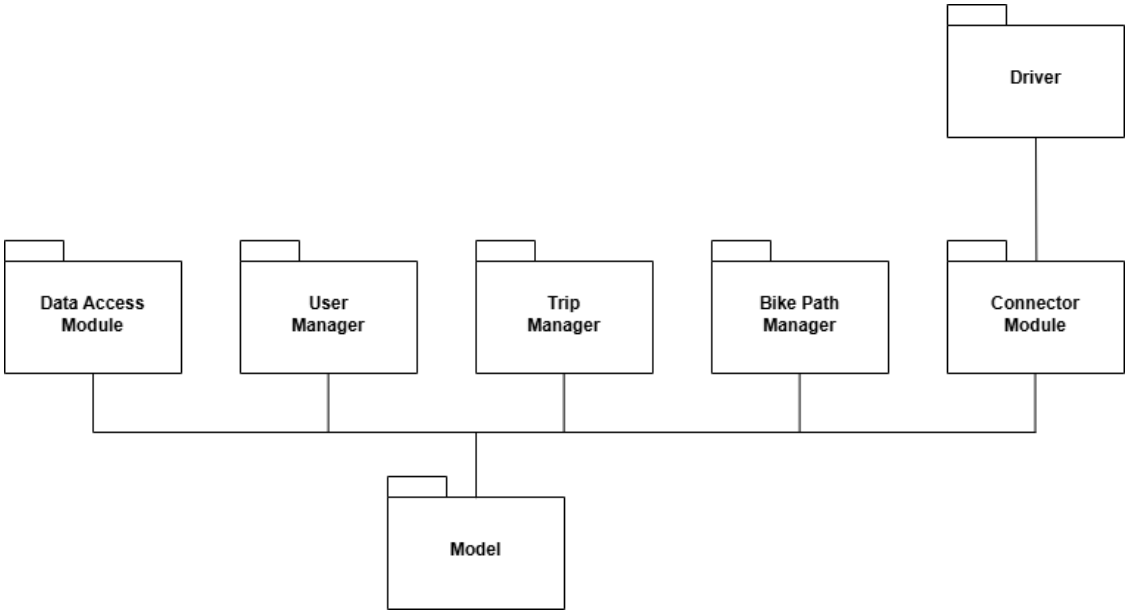


Figure 5.4: Implementation step: 4

5.2.5. Report Manager

This component must be implemented in this step since its function is a precursor for the next modules. It shall be tested isolated using drivers that simulate the reports arriving from the users, then it will be integrated and tested with the system.

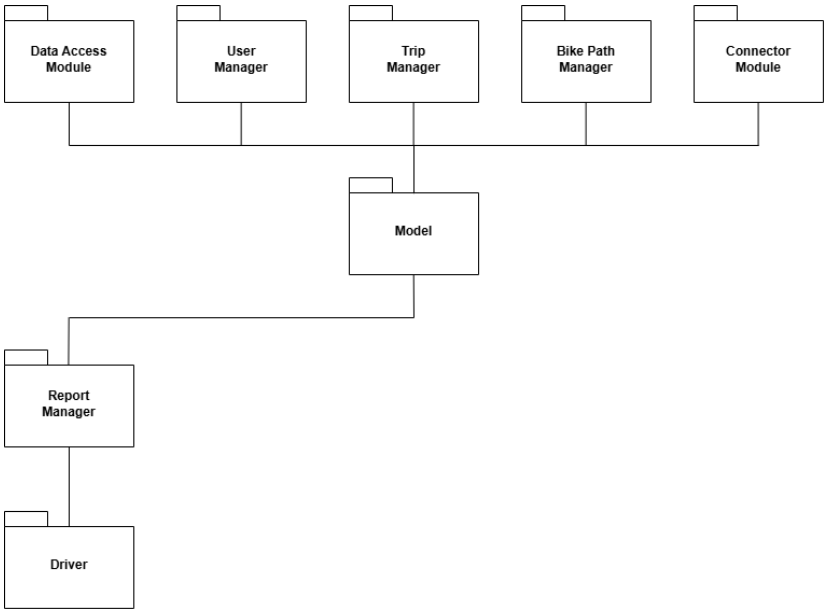


Figure 5.5: Implementation step: 5

5.2.6. Street Manager

This component interact with the previous one and it's responsible to keep track of streets' updates and to propagate eventual changes. First, it gets implemented and tested using drivers that simulate the Reports. Then, it shall be integrated and tested with the other components.

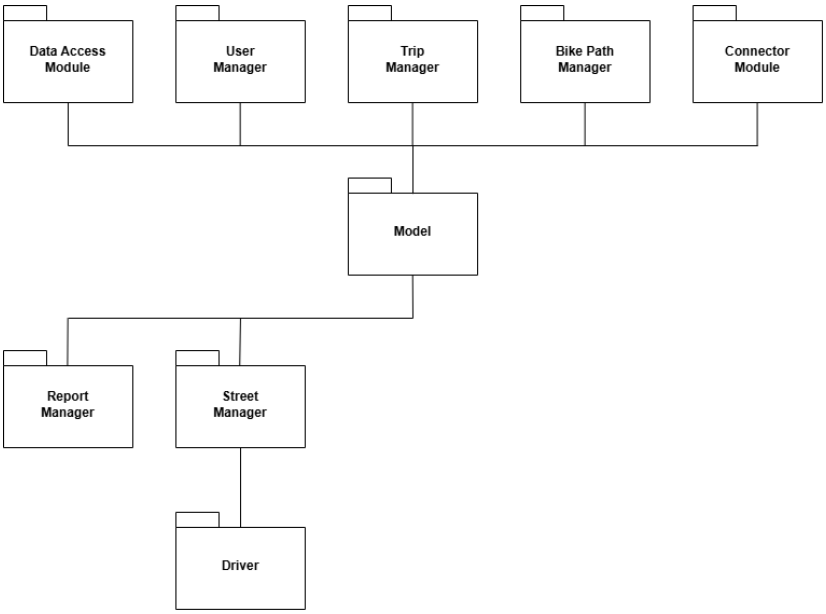


Figure 5.6: Implementation step: 6

5.2.7. Path Recommendation Engine

This component will handle users' researches on the platform and it consults the other modules and the database to retrieve the best paths. It shall be first implemented and tested through drivers. Then, it will be integrated and tested with the other components.

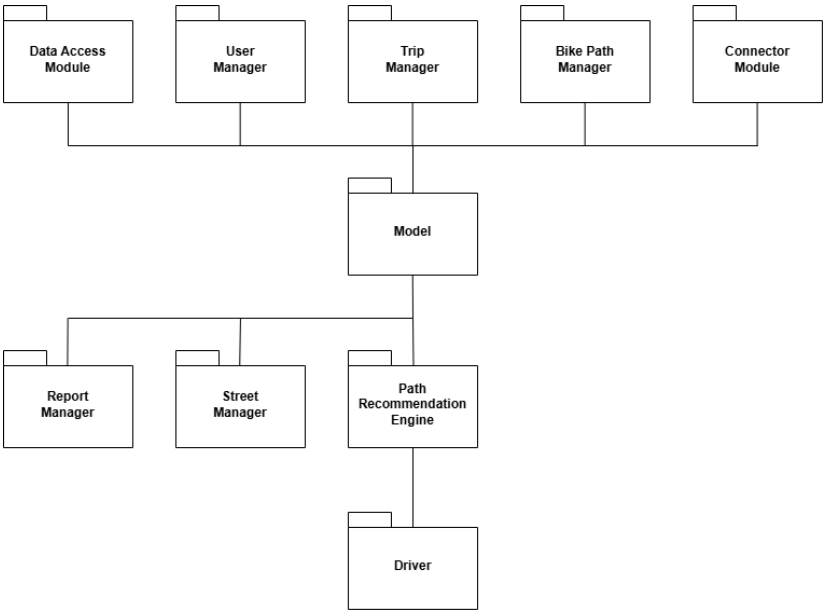


Figure 5.7: Implementation step: 7

5.2.8. Sensor Manager

This components is responsible to retrieve data from the client device. It will first be implemented and tested individually, then it will be integrated with the other components and it will be tested repeating some previous tests where drivers have been used.

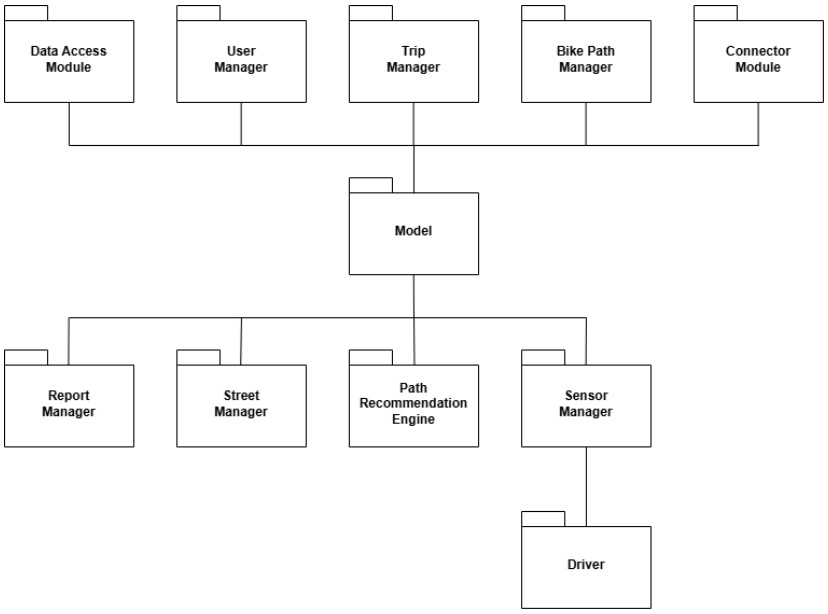


Figure 5.8: Implementation step: 8

5.2.9. Trip Detector

This component is implemented only at this step since it's an enhancement of the user experience. It shall be implemented and tested isolated first, then integrated with the other modules on the device, and later tested with the modules on the server.

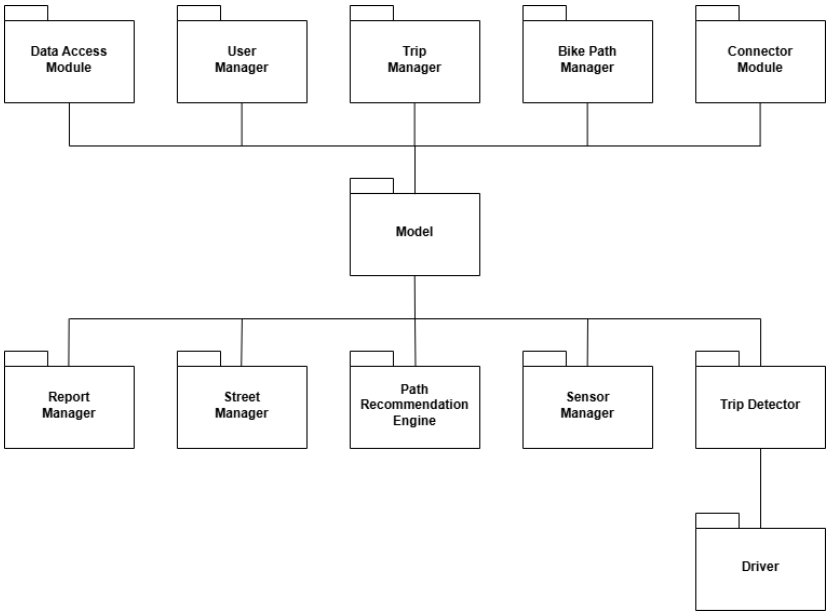


Figure 5.9: Implementation step: 9

5.2.10. Metrics Engine

This one too is a medium priority component to be implemented and tested, first using some drivers. Only later, it will be integrated and tested with the system.

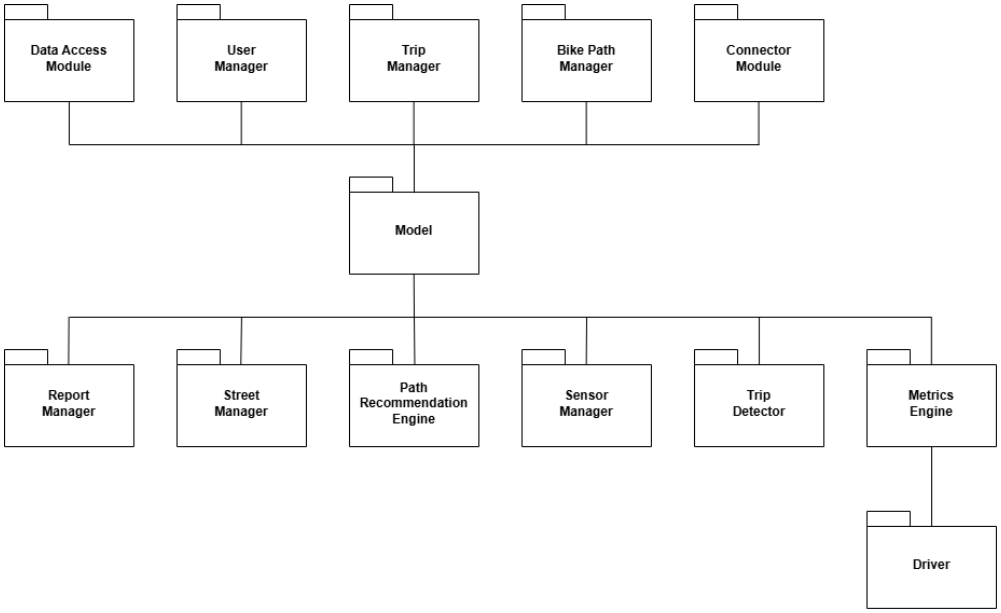


Figure 5.10: Implementation step: 10

5.2.11. External API Manager

This is one of the low priority components to be implemented since its feature is to enrich the experience and it's easy to implement. Once it's been tested alone, it can be integrated and tested with the rest of the system.

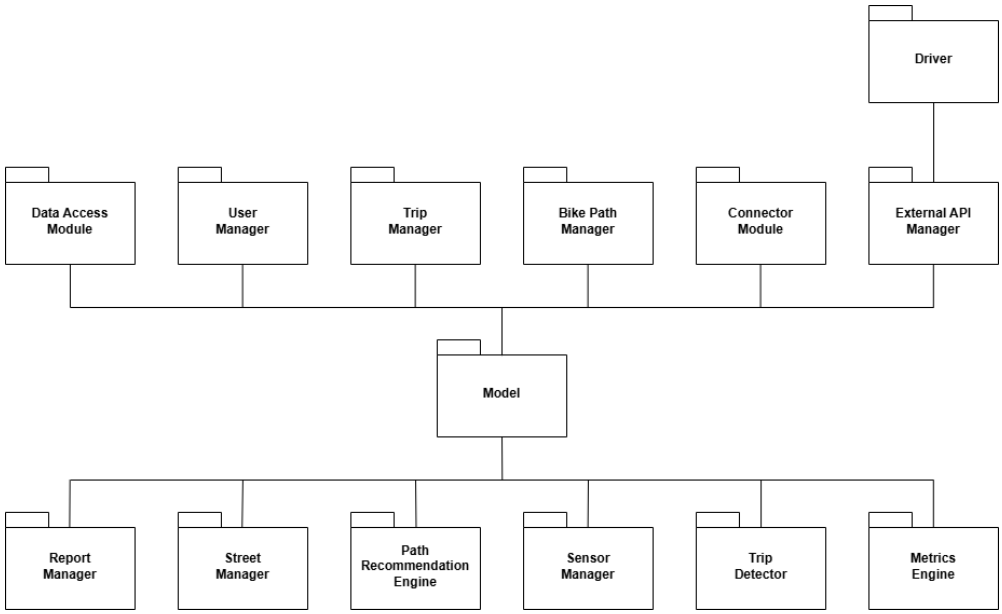


Figure 5.11: Implementation step: 11

5.2.12. Offline Sync Module

This one is an extra component designed to enhance the user experience in case of external difficulties. Once it's been implemented and tested isolated, it shall be integrated with the system and finally test the whole system.

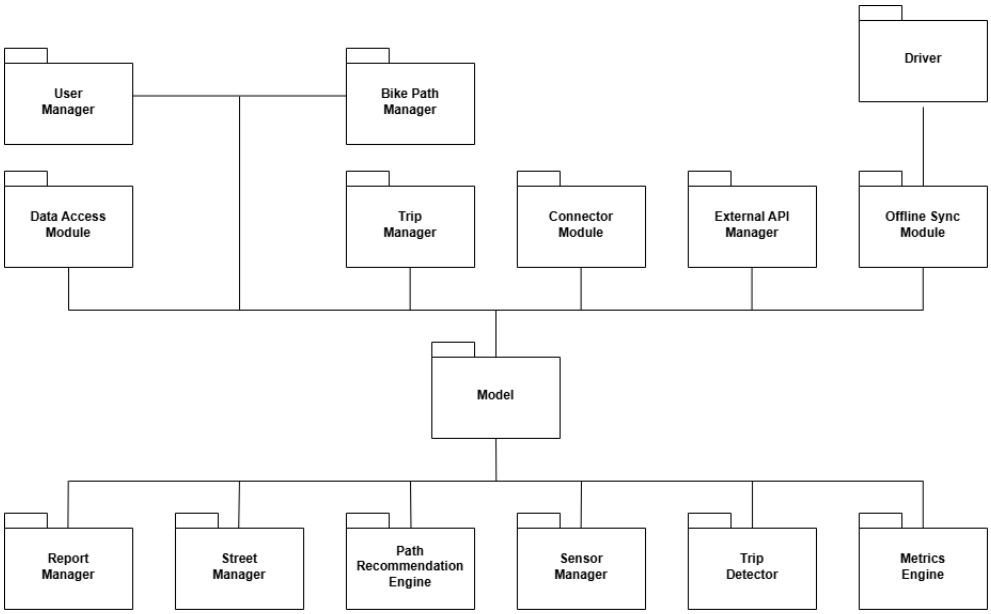


Figure 5.12: Implementation step: 12

5.3. System Testing

After completing the development and integration phases with their relative tests, the platform will undergo **system testing**:

- Use cases from the RASD document will be used to verify that all functionalities work as expected. Including verification on both functional and non-functional requirements.
- **Performance testing**: All the system (client/server) will be tested in order to find any bottlenecks and guarantee that all the performance requirements from the RASD document are met. The client side of the system will be also tested on different devices (real/emulated) in order to ensure that the client side works fine on several devices.
- **Stress testing**: The system will be tested simulating lot of concurrent requests to ensure that the platform handles high loads in accordance to the estimates provided in the RASD document.
- **User interface testing**: The system UI will undergo testing on a wide variety of mobile devices in order to guarantee a correct UI rendering on different screen sizes.

6 | Effort spent

Task	Simone Controguerra	Matteo Delogu	Gianluca Di Paola
Chapter 1	2 hours	2 hours	2 hours
Chapter 2	4 hours	3 hours	2 hours
Chapter 3	- hours	- hours	- hours
Chapter 4	2 hours	2 hours	4 hours
Chapter 5	5 hours	5 hours	4 hours
Total Effort	0 hours	0 hours	0 hours

Table 6.1: Effort Spent by Each Group Member

7 | References

- Microservice Pattern: https://microservices.io/patterns/index.html

8 | Use of Generative AI

- **Writefull for Overleaf:** GEN-AI integration for Overleaf offering real time language feedback.